EX.NO.: 11
DATE: 04.03.2025

FORMATING GENERATED IMAGE

AIM
To develop a simple and user-friendly image editing tool that allows users to enhance and customize their images.

TECHNIQUES USED
1. **Graphical User Interface (GUI) Development**
    a. Built using **Tkinter** to provide an interactive and user-friendly interface.
    b. Organized layout with sliders and entry fields for user input.
2. **Image Processing**
    a. **PIL (Pillow)**: Used for handling images, enhancing sharpness, and applying watermarks.
    b. **OpenCV (cv2)**: Used for noise reduction through non-local means denoising.
3. **Enhancement Methods**
    a. **Sharpness Adjustment**: Implemented using ImageEnhance.Sharpness from Pillow.
    b. **Noise Reduction**: Utilized OpenCV's fastNlMeansDenoisingColored() function.
    c. **Watermarking**: Text overlay is applied using ImageDraw.Draw().
4. **File Handling & Exporting**
    a. Users can load images from their local storage.
    b. Images can be saved in different formats (PNG, JPEG, TIFF) with applied modifications.

CODE AND OUTPUT
```
import tkinter as tk
from tkinter import filedialog, ttk
from PIL import Image, ImageEnhance, ImageTk, ImageDraw
import cv2
import numpy as np

class ImageEditor:
    def __init__(self, root):
        self.root = root
        self.root.title("Image Editor")
        self.root.geometry("600x600")

        # Main Frame for Layout
        self.main_frame = tk.Frame(self.root)
        self.main_frame.pack(pady=10, padx=10, fill=tk.BOTH, expand=True)

        # Image Display
        self.image_label = tk.Label(self.main_frame, text="No Image Loaded",
relief=tk.SUNKEN, width=60, height=15)
        self.image_label.grid(row=0, column=0, columnspan=2, pady=10)

        # Load Image Button
        self.load_button = ttk.Button(self.main_frame, text="Load Image",
command=self.load_image)
        self.load_button.grid(row=1, column=0, columnspan=2, pady=5)

        # Feature Adjustments Frame
```

```
        self.controls_frame = tk.Frame(self.main_frame)
        self.controls_frame.grid(row=2, column=0, columnspan=2, pady=10, padx=10)

        # Sharpness Adjustment
        ttk.Label(self.controls_frame, text="Sharpness").grid(row=0, column=0, padx=5,
pady=5)
        self.sharpness_scale = ttk.Scale(self.controls_frame, from_=0, to=3,
orient=tk.HORIZONTAL, length=200)
        self.sharpness_scale.grid(row=0, column=1, padx=5, pady=5)

        # Noise Reduction
        ttk.Label(self.controls_frame, text="Noise Reduction").grid(row=1, column=0,
padx=5, pady=5)
        self.noise_reduction_scale = ttk.Scale(self.controls_frame, from_=0, to=10,
orient=tk.HORIZONTAL, length=200)
        self.noise_reduction_scale.grid(row=1, column=1, padx=5, pady=5)

        # Watermark Input
        ttk.Label(self.controls_frame, text="Watermark Text").grid(row=2, column=0,
padx=5, pady=5)
        self.watermark_entry = ttk.Entry(self.controls_frame, width=25)
        self.watermark_entry.grid(row=2, column=1, padx=5, pady=5)

        # Apply Enhancements Button
        self.apply_button = ttk.Button(self.main_frame, text="Apply Enhancements",
command=self.apply_enhancements)
        self.apply_button.grid(row=3, column=0, pady=10)

        # Save Image Button
        self.save_button = ttk.Button(self.main_frame, text="Save Image",
command=self.save_image)
        self.save_button.grid(row=3, column=1, pady=10)

        self.image = None
        self.processed_image = None

    def load_image(self):
        file_path = filedialog.askopenfilename(filetypes=[("Image Files",
"*.png;*.jpg;*.jpeg;*.tiff")])
        if file_path:
            self.image = Image.open(file_path)
            self.display_image(self.image)

    def display_image(self, image):
        image.thumbnail((300, 300))
        img_tk = ImageTk.PhotoImage(image)
        self.image_label.config(image=img_tk, text="")
        self.image_label.image = img_tk
```

```
    def apply_enhancements(self):
        if not self.image:
            return

        self.processed_image = self.image.copy()

        sharpness_factor = self.sharpness_scale.get()
        enhancer = ImageEnhance.Sharpness(self.processed_image)
        self.processed_image = enhancer.enhance(sharpness_factor)

        noise_factor = self.noise_reduction_scale.get()
        if noise_factor > 0:
            np_image = np.array(self.processed_image)
            np_image = cv2.fastNlMeansDenoisingColored(np_image, None, noise_factor *
5, noise_factor * 5, 7, 21)
            self.processed_image = Image.fromarray(np_image)

        watermark_text = self.watermark_entry.get()
        if watermark_text:
            draw = ImageDraw.Draw(self.processed_image)
            draw.text((10, 10), watermark_text, fill="white")

        self.display_image(self.processed_image)

    def save_image(self):
        if not self.processed_image:
            return

        file_path = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[
            ("PNG", "*.png"),
            ("JPEG", "*.jpg"),
            ("TIFF", "*.tiff")
        ])
        if file_path:
            self.processed_image.save(file_path)

if __name__ == "__main__":
    root = tk.Tk()
    app = ImageEditor(root)
    root.mainloop()
```

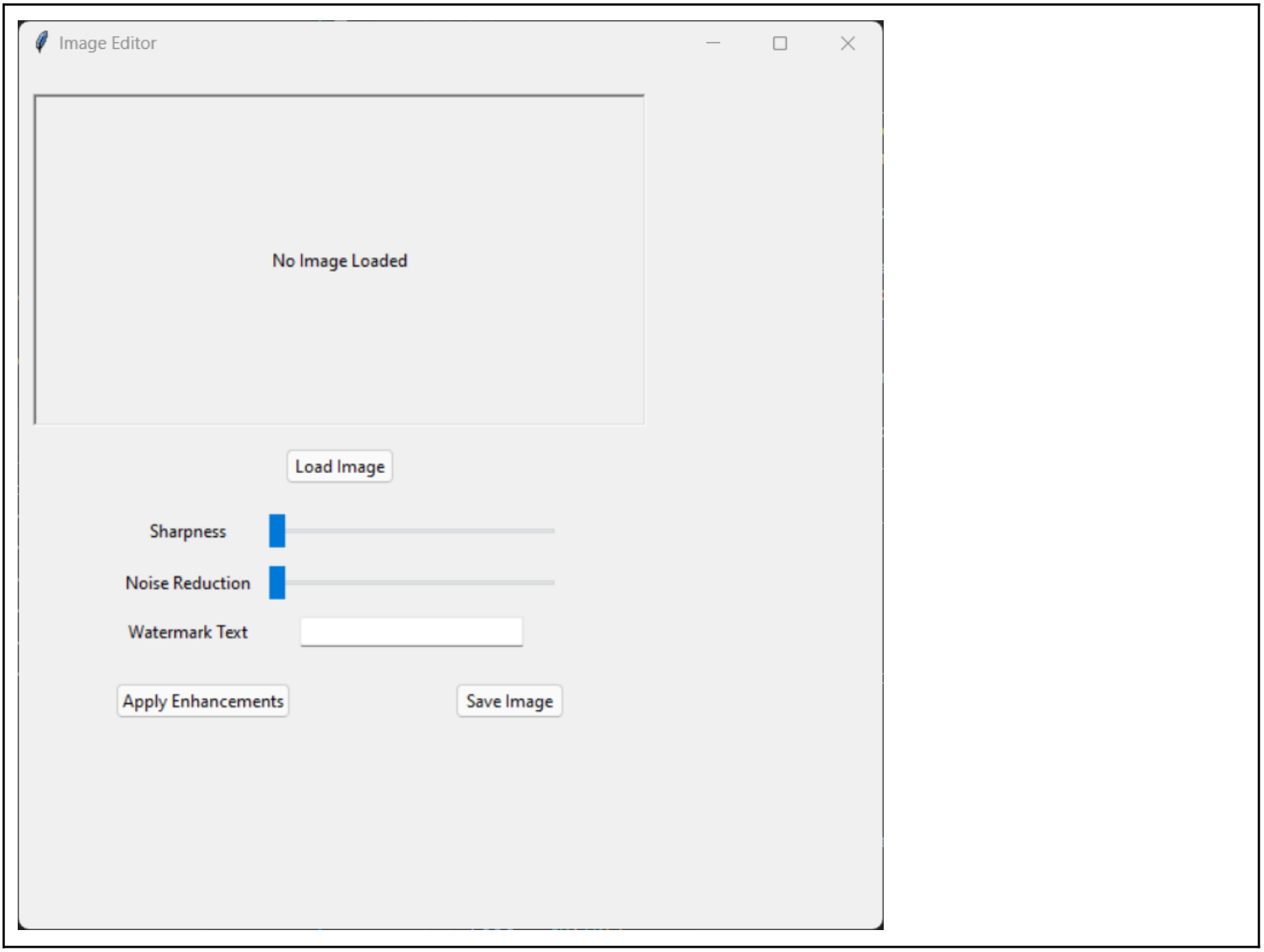
Image Editor
No Image Loaded
Load Image
Sharpness
Noise Reduction
Watermark Text
Apply Enhancements
Save Image